大语言模型的训练

第 9 章（下）

第 14 课时 本课时讲解 9.2（后半）、9.3–9.6

训练显存与并行 · 混合并行策略 · 分布式训练系统

著者
邓杭　杨耀东

教材
《大语言模型：训练、精调与对齐》

出版
清华大学出版社



本章目录 第 14 课时

9.1 大语言模型训练
三大挑战、训练框架、显存开销与通信基础 第 3 页

9.2 混合并行策略
四种并行及其正交组合 第 24 页

9.3 分布式训练系统
DeepSpeed 零冗余优化器与显存分区 第 61 页

9.4 小结
并行算法的取舍与系统视角 第 75 页

9.5 附录：专家并行*
MoE 的专家部署与两次全交换 第 79 页

9.6 附录：训练-推理解耦架构*
RL 训练的采样部署、参数同步与异步推理 第 84 页

本课时内容 9.2 混合并行策略（后半）、9.3 分布式训练系统、9.4 小结、9.5 附录：专家并行*、9.6 附录：训练-推理解耦架构*（其余部分见第 13 课时）

9.2

混合并行策略

核心挑战在于执行远超单个设备显存容量的计算任务，同时高效利用集群资源；四种并行维度彼此正交，可以组合成 4D 并行。

本节内容

9.2.1 数据并行

9.2.2 张量并行

9.2.3 流水线并行

9.2.4 上下文并行

9.2.5 本节小结

流水线并行

除了张量并行，还可以在完整模型层面上将其切分成多个阶段，这种切分方法称为流水线并行。与张量并行单个 GPU 只持有参数矩阵的一部分不同，流水线并行中每个 GPU 持有其对应阶段完整的参数矩阵。

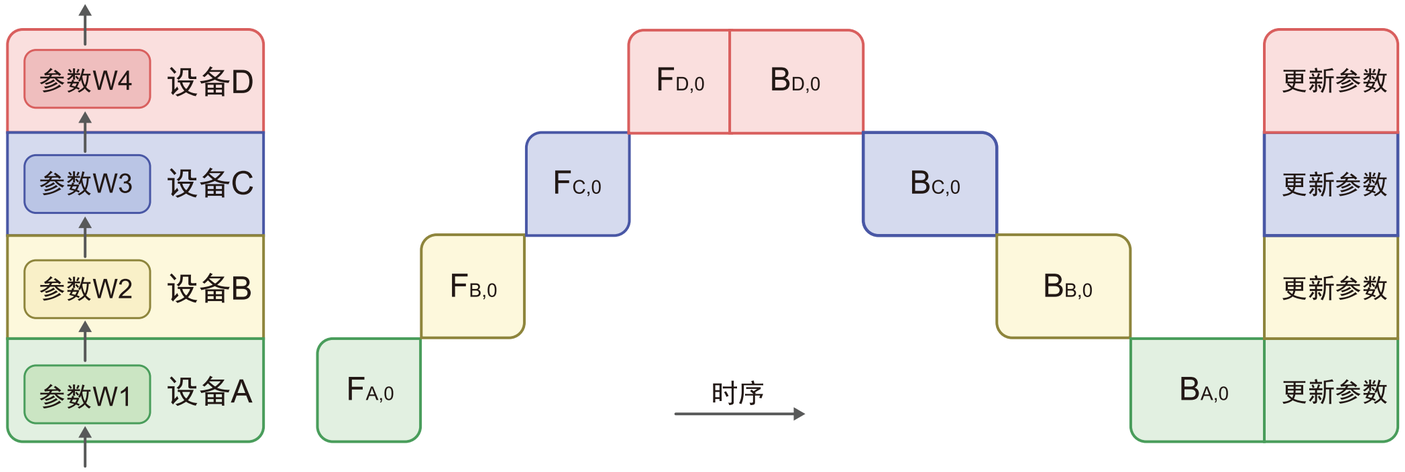


图 9.19　流水线并行中的气泡现象及其对集群利用率的影响

示例

8 层模型，4 个阶段

每两层划分为一个阶段，4 个阶段分配到 4 个 GPU 上。每个 GPU 只需存储和计算其分配阶段所需的参数和激活值。

这显著降低了单个 GPU 的显存需求，使得可以用更多 GPU 来训练更深的模型。

GPipe 提出的流水线并行方法，能够有效减少训练时单卡的显存占用。代价是设备之间必须串行传递激活值与梯度，从而产生等待。

图 《大语言模型：训练、精调与对齐》图 9.19；第 9.2.3 节

流水线的气泡

在这种训练模式下，设备间存在等待时间。例如设备 A 是第一个完成前向计算的，但却是最后一个进行反向梯度计算的，这导致它在前向结束后需要等待很长时间才能开始反向计算。

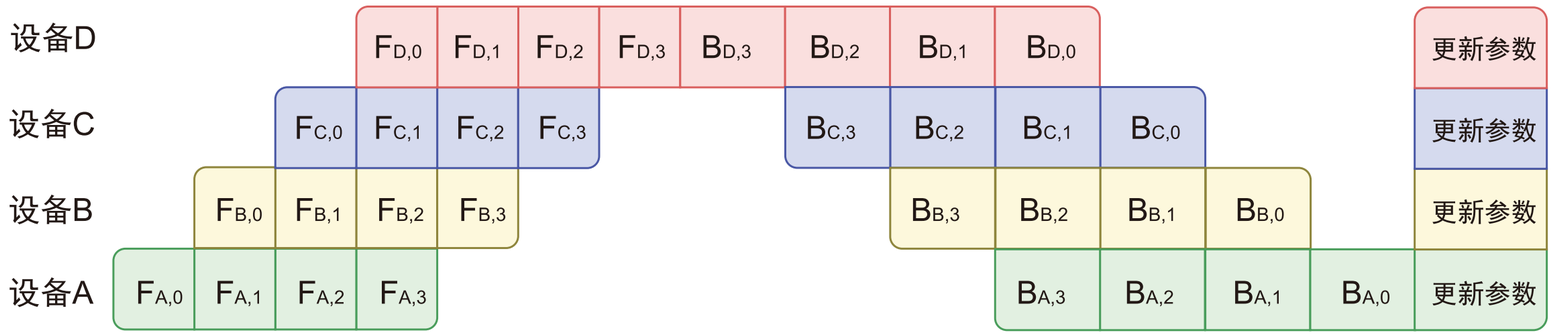
定义 多 GPU 系统中设备因等待通信而处于空闲状态的时间称为气泡。流水线并行系统中气泡越少，则系统的整体计算利用率越高。

$$\text{气泡时间占比} = \frac{\text{所有 GPU 的空闲等待时间之和}}{\text{整个流水线过程的总运行时间}}$$

这是衡量气泡大小的指标：分母包含所有阶段的计算与通信，分子只统计空转。

沿用图 9.19 的记号：P 个阶段、每个批量切成 M 个微批量时，气泡占比约为 $(P-1)/(P-1+M)$。阶段数越多、微批量越少，空转越显著——这正是 GPipe 引入微批量、PipeDream 改用异步调度的原因。

GPipe



核心

批量再细分

GPipe 提出微批量的概念来配合编排流水线调度执行任务：将一个批量数据细分为多个小的计算单元。

图 9.20　微批量处理通过流水线调度减少气泡率

前向调度

① 第 1 个 GPU 先算 $\mathrm{F}_{A,0}$；② 把结果传给第 2 个 GPU；③ 下一时间步，第 1 个 GPU 算 $\mathrm{F}_{A,1}$，同时第 2 个 GPU 处理 $\mathrm{F}_{B,0}$；④ 持续进行直至所有微批量完成前向。

反向调度

① 第 4 个 GPU 先算 $\mathrm{B}_{D,3}$；② 把梯度传给第 3 个 GPU；③ 下一时间步，第 4 个 GPU 算 $\mathrm{B}_{D,2}$，第 3 个 GPU 处理 $\mathrm{B}_{C,3}$；④ 全部完成后各阶段同步更新参数。

图 《大语言模型：训练、精调与对齐》图 9.20；第 9.2.3 节

微批量的代价

微批量处理的核心价值在于减少气泡时间占比：通过精细的流水线调度，使更多设备保持并行计算状态。理论上，微批量数量越多，气泡占比越低，系统效率越高。

收益：气泡变小

更多设备在同一时刻有活可干，硬件利用率上升。

气泡占比大致随微批量数量的增加而下降。

代价：激活值变多

增加微批量数量会大幅增加激活值的存储需求，显存压力随之上升。

在显存与气泡之间需要折中，而不是一味加大微批量数。

缓解 采用重计算策略：前向传播中丢弃部分激活值，反向传播时按需重新计算。该技术称为梯度检查点。

简单激活

对于 ReLU 等计算简单的激活函数，可仅保留激活前的值，重计算开销较小，但可减少约 50% 的显存占用。

复杂层

计算复杂的层可保留激活值，避免重计算开销，把省显存的任务交给代价更低的位置。

PipeDream

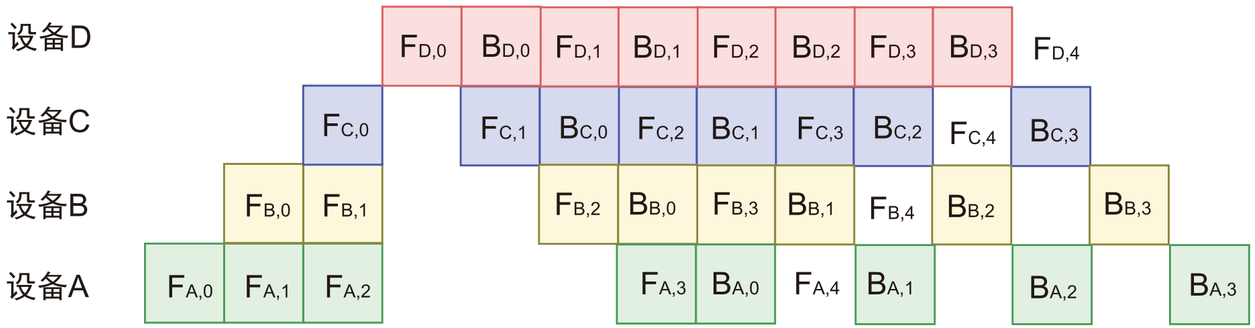


图 9.21　PipeDream 的异步流水线并行策略

- GPipe 中所有设备同步进行参数更新；PipeDream 采用异步更新策略，允许不同设备在不同时间更新参数。
- 通过在单个 GPU 内交错执行前向传播和反向传播计算（通常称这种复合操作为 1F1B），最大限度减少硬件空闲时间。
- 完成微批量计算后立即释放激活值，显著降低显存需求。

阶段 1

预热

逐步填充流水线，主要做前向。

阶段 2

稳定

流水线充满，设备同时在算。

图 《大语言模型：训练、精调与对齐》图 9.21；第 9.2.3 节

异步流水线的问题

当设备 A 处理第 5 个微批量数据（$\mathrm{F}_{A,4}$）时，各设备上的模型参数已基于不同数据批次进行了更新。

现象

版本分叉

设备 A 的参数仅使用数据 1 的梯度更新过，而设备 D 的参数已使用数据 1–4 的梯度进行了四次更新。同一时刻不同阶段的参数版本不一致。

对策

参数快照

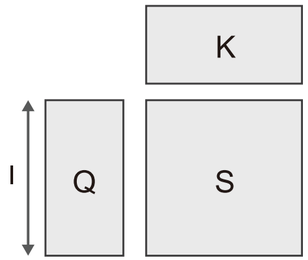
为确保梯度计算正确性，PipeDream 需要为每个微批量存储对应的参数快照，这会显著增加存储开销。

主流 1F1B 实现会采用梯度累计技术：并非计算完梯度就更新参数，而是达到更新条件再进行参数同步，保证计算时前向与反向的参数版本唯一。

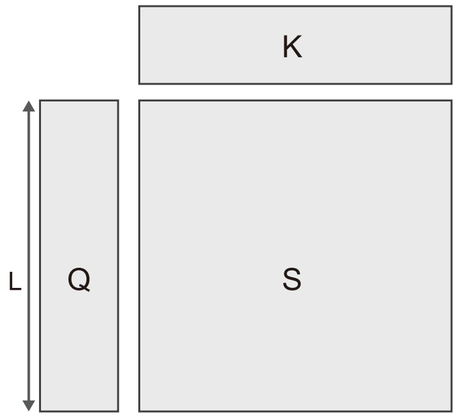
后续研究进一步优化了流水线并行：Chimera 提出双流水线并行方法；ZeroBubble 提出将参数更新与输入梯度传递操作异步并行执行，该复合操作被称为 1F1B1W；DeepSeek-V3 提出的 DualPipe 通过重叠计算与通信操作，有效隐藏了专家并行中全交换通信的时间开销。

从 1F1B 到 1F1B1W 再到 DualPipe，改进的方向始终一致：把更多的操作塞进同一段时间里，让等待消失。

长序列的计算代价



(a) 注意力分数 1



(b) 注意力分数 2

图 9.22(a)(b)　块注意力计算示意图：序列变长，分数矩阵迅速变大

长篇小说、工程代码等语言数据包含较长的文本。随着序列长度 L 的线性增长，注意力计算的时间和空间复杂度呈平方级增长：

$$\mathcal{O}(L^2)\ \text{的时间与空间} \quad \Longrightarrow \quad \text{单 GPU 不再可行}$$

这使得在单个 GPU 上处理更长序列的注意力计算变得不可行。

上下文并行

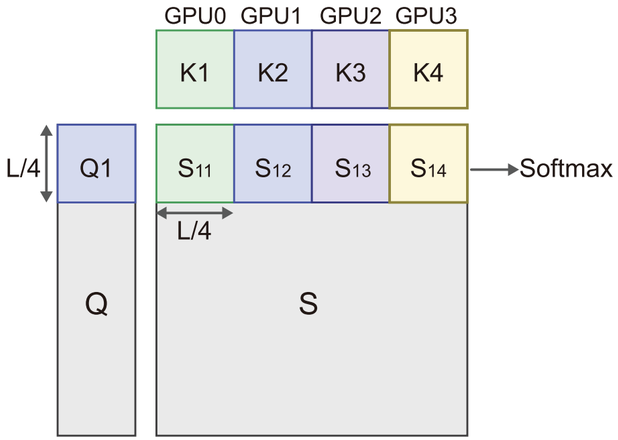


图 9.22(c)　分块注意力分数

一种解决方案是在序列长度维度上切分数据，称为序列并行或上下文并行：每个 GPU 仅接收并存储一个子序列片段。

本地计算

局部注意力

每个 GPU 只能基于其本地片段进行局部注意力计算，注意力分数矩阵被切成若干块。

核心挑战

如何规约

如何高效地将分布在多个 GPU 上的局部注意力结果规约起来，形成最终的全局注意力输出。

前馈层的计算在序列维度上完全独立，不会引入额外通信开销。

图 《大语言模型：训练、精调与对齐》图 9.22(c)

上下文并行的难点

在计算注意力前，将词向量投影成查询矩阵 $\boldsymbol{Q}$、键矩阵 $\boldsymbol{K}$ 和值矩阵 $\boldsymbol{V}$。在上下文并行方案中，将 $\boldsymbol{Q}$、$\boldsymbol{K}$、$\boldsymbol{V}$ 沿序列维度切分，并把不同的 KV 块分配到不同的 GPU 上。

$$P_{ij} = \frac{\exp(S_{ij})}{\sum_{k=1}^{L} \exp(S_{ik})}$$

由于 Softmax 函数的非线性特性，计算全局归一化的注意力分数面临挑战：需要规约所有 GPU 上的局部注意力分数信息来计算全局 Softmax 分母项，再将归一化因子分发给各 GPU。

本地分数

各卡算本地 S_{ij}

→

规约分母

跨卡收集 $\exp(S_{ik})$ 的和

→

分发因子

把全局分母发回各卡

→

本地输出

归一化后加权求和

这个过程会引入显著的通信开销——每一步都要等所有卡算完才能继续，正好与流水线中的气泡同源。

依据 《大语言模型：训练、精调与对齐》第 9.2.4 节

环形注意力

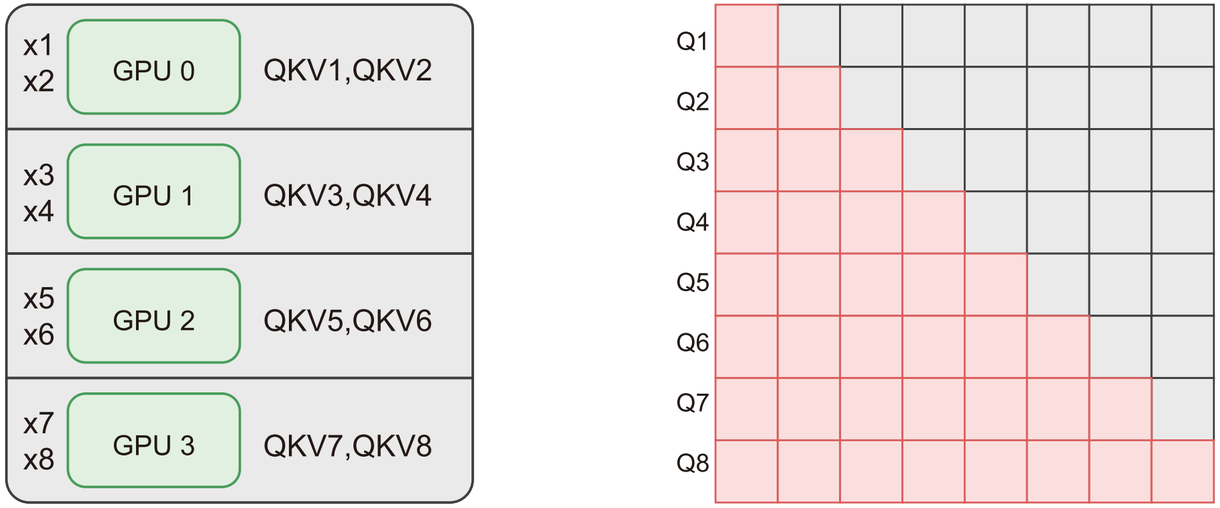


图 9.23　KV 序列切分

借助 Flash Attention-V2 的分块注意力计算思想，采用环形注意力算法实现分布式环境的分块注意力计算。初始时，每个 GPU 仅有本地的 KV 块。

- 每个 GPU 加载一个固定的 $\boldsymbol{Q}$ 块。
- 算法通过环形通信拓扑，在多个步骤间依次传递 $\boldsymbol{K}$、$\boldsymbol{V}$ 块。
- 每个步骤中，GPU 利用当前可用的 KV 块计算部分注意力结果，并逐步累积更新最终的注意力输出 $\boldsymbol{O}$。

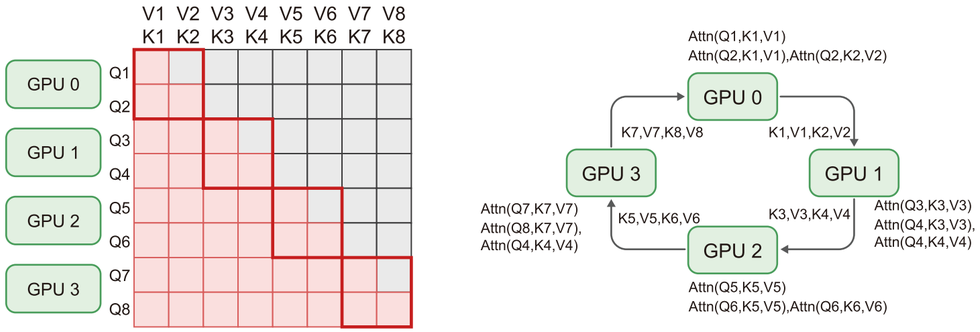
$$N \text{ 个 GPU：} \quad N-1 \text{ 次环形传递} + N \text{ 次计算} \quad \Longrightarrow \quad \text{每卡得到完整输出}$$

环状注意力在数学上与标准的注意力计算等价。

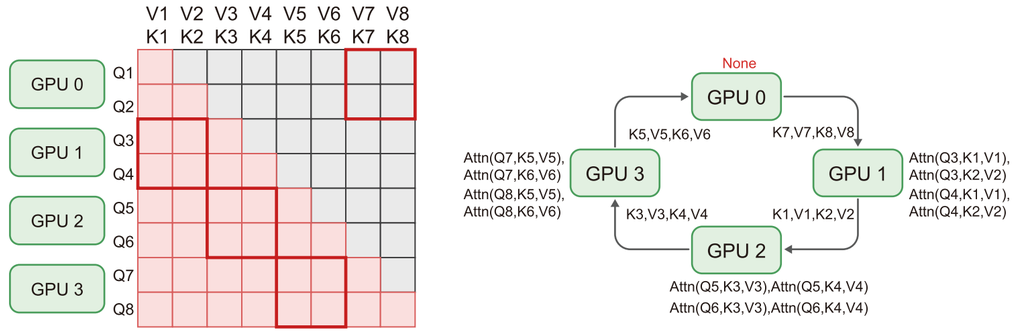
图 《大语言模型：训练、精调与对齐》图 9.23；第 9.2.4 节

环形注意力的计算过程

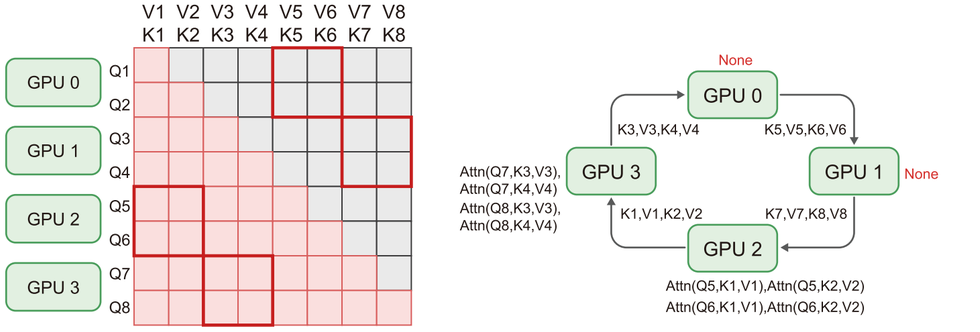
$N = 4$ 时共 4 个计算步骤：每个 GPU 持有固定的 $\boldsymbol{Q}_i$，每一轮把 $\boldsymbol{K}, \boldsymbol{V}$ 环传给后继，同时用收到的块累加本地的部分输出与归一化因子。第 3 步结束时，$\boldsymbol{Q}_i$ 已看过全部 KV，累积结果就是完整的 $\boldsymbol{O}_i$。



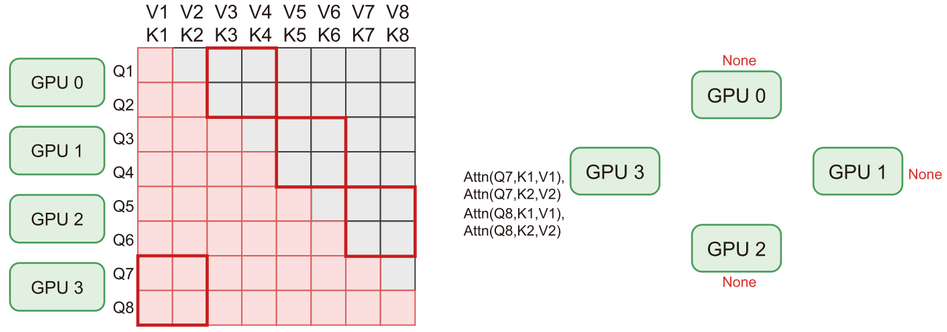
(a) 第 0 步　各卡只用本地 $\boldsymbol{K}_i, \boldsymbol{V}_i$



(b) 第 1 步　传递一轮后累加前驱的块



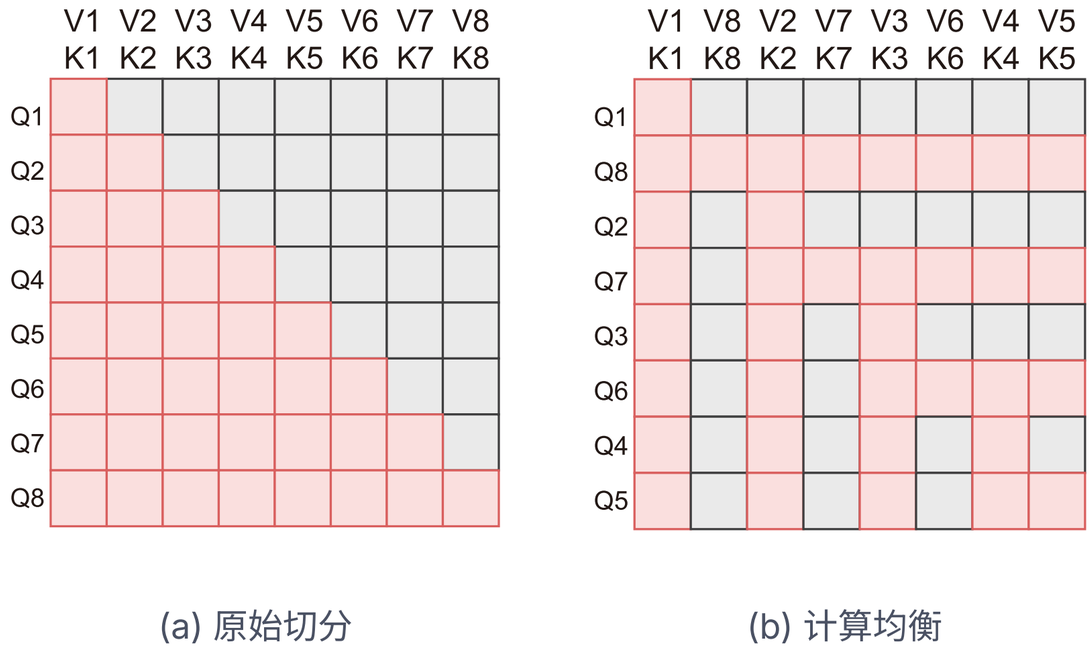
(c) 第 2 步　已看过一半的 KV



(d) 第 3 步　$\boldsymbol{Q}_i$ 看过全部 KV，输出完整

图 《大语言模型：训练、精调与对齐》图 9.24

负载不均与序列置换



(a) 原始切分　　(b) 计算均衡

图 9.25　环形注意力计算负载均衡

对于使用因果掩码的自回归训练，各 GPU 的块注意力计算量是不同的，这可能导致计算负载不均衡和硬件利用率 MFU 下降。

- 原始切分下，$(\boldsymbol{Q}_1, \boldsymbol{Q}_2)$ 块与 $(\boldsymbol{Q}_7, \boldsymbol{Q}_8)$ 块的注意力计算量显著不同：靠后的块能看到的键更多。
- 采用序列置换策略后，$(\boldsymbol{Q}_1, \boldsymbol{Q}_8)$ 与 $(\boldsymbol{Q}_2, \boldsymbol{Q}_7)$ 的注意力计算量一致。
- 从而在整体上，可以减少接近一半的原始块注意力计算量。

实质是让每个设备同时持有一段“靠前”和一段“靠后”的序列，把长短不一的三角形工作量拼成相等的两块。

图 《大语言模型：训练、精调与对齐》图 9.25；第 9.2.4 节

四种并行的选择

并行方式	切分对象	通信频率	适用场景
数据并行	数据样本	低（每步梯度同步）	通用首选，可跨节点
张量并行	层内参数矩阵	高（前后向均插入）	模型参数更大，需更细粒度切分
流水线并行	层深度	中（阶段边界传激活与梯度）	通信性能较差的场景更有效
上下文并行	序列长度	中（KV 环形传递）	训练序列更长时必需

这些策略可以正交组合，以适应不同规模的模型参数和序列长度需求。当模型参数更大时，需要采用更细粒度的模型并行；当训练序列更长时，则需要引入上下文并行机制。

开放问题

自动并行

如何自动地寻找最优的并行切分策略，是分布式训练系统的关键研究问题之一；同时优化计算效率和减少通信开销也是持续的努力方向。

边界

不减少总计算量

上述并行方法主要解决分布式计算的问题，并未减少模型的总计算量。如何从根源上降低计算量，特别是注意力的平方复杂度，仍是模型设计面临的重大挑战。

9.3

分布式训练系统

ZeRO 用增加通信换取显存：把参数、梯度与优化器状态分区，让每张卡只保留完整状态的 $1/N$。

本节内容

9.3.1　DeepSpeed 零冗余优化器（ZeRO）

显存占用分析

三个优化阶段

参数切分与流程

ZeRO 方法

DeepSpeed 是一个通用的分布式深度学习训练框架，旨在显著降低显存占用。其核心算法是零冗余优化器（Zero Redundancy Optimizer, ZeRO）。

传统数据并行

每个 GPU 设备都存储完整的模型参数、梯度和优化器状态副本。

数据不同、副本相同——同样的状态在 N 张卡上重复了 N 份。

ZeRO

将这些模型状态（权重参数、梯度和优化器状态）划分并分布在集群中可用的 GPU 或 CPU 上。

每张卡只保留自己负责的那一片，需要时再通过通信取回。

> **定位** ZeRO 是对数据并行的一种扩展实现：计算的组织方式仍是数据并行，改变的是状态的存放方式。

模型状态的显存占用



图 9.26　训练过程显存占用

模型状态	精度	字节数
模型参数	FP16	2Ψ
梯度	FP16	2Ψ
优化器状态（FP32 参数副本）	FP32	4Ψ
优化器状态（动量）	FP32	4Ψ
优化器状态（方差）	FP32	4Ψ
合计	—	16Ψ

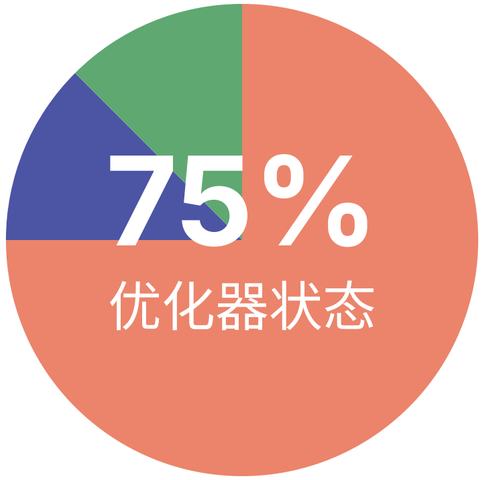
$$2\Psi + 2\Psi + 12\Psi = 16\Psi \text{ 字节}, \qquad 16 \times 7.5 \times 10^9 = 120 \text{ GB}$$

以参数量为 Ψ 的模型配合 Adam 优化器进行混合精度训练为例。7.5B 参数量的模型，其模型状态至少需要 120 GB 显存空间。

除模型状态外，显存还包含激活值、中间计算变量以及显存碎片；中间变量包含通信操作（如 All-Gather）所需的存储开销。数据并行度为 N 时，这些相同的模型状态会在 N 个 GPU 上复制 N 份。

优化器状态的显存占比

在模型状态中，优化器状态通常是显存占用最大的部分。ZeRO 的核心思想之一，正是通过优化器状态分区来消除这部分冗余。



模型状态 16Ψ 的组成（并行度 1）

组成部分	每参数数量	占比
优化器状态	12Ψ	75%
梯度	2Ψ	12.5%
参数	2Ψ	12.5%

120 GB

7.5B 模型的状态总量

75%

优化器状态占比

1.9 GB

ZeRO-3 每卡占用

优化器状态由动量、方差与 FP32 参数副本三份构成；6Ψ 到 16Ψ 的差距全部来自这三份副本。把它们切开，收益远大于切梯度或切参数本身。

依据 《大语言模型：训练、精调与对齐》第 9.3.1 节

ZeRO 的三个阶段

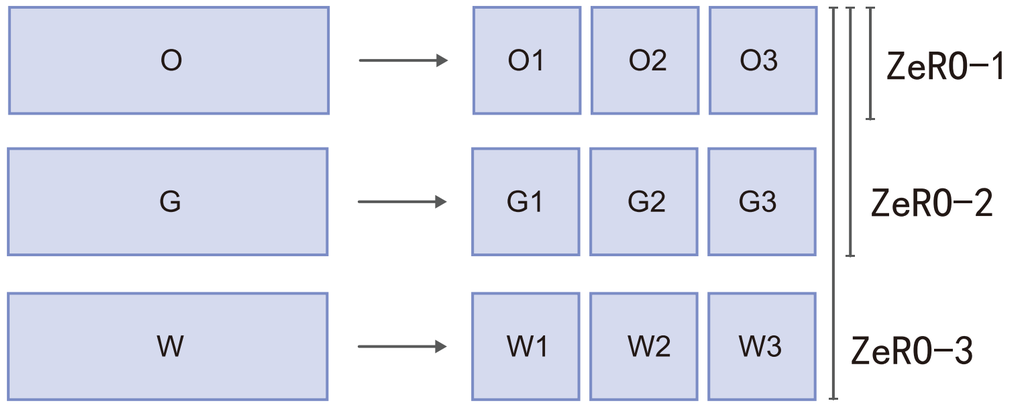


图 9.27　ZeRO 优化的 3 个阶段

- **ZeRO-1**：切分优化器状态，包括动量和方差。
- **ZeRO-2**：在 ZeRO-1 基础上增加梯度切分，确保每个 GPU 上的优化器状态分区与梯度分区相互对应。
- **ZeRO-3**：进一步切分模型参数本身。

需要注意的是，当参数切分未遵循张量并行的行列规则时，无法直接在局部参数分区上执行矩阵乘法运算，需通过通信操作收集参数。

图 《大语言模型：训练、精调与对齐》图 9.27；第 9.3.1 节

ZeRO 三个阶段的通信量

ZeRO 在数据并行基础上引入参数分区机制。应用 ZeRO-3 后，每个 GPU 仅存储完整模型状态的 $1/N$（N 为 GPU 数量），各分区互不重叠，完全消除冗余存储。

阶段	切分内容	每卡状态量	新增通信
基线 DP	不切分	16Ψ	All-Reduce 梯度
ZeRO-1	优化器状态	$4\Psi + 12\Psi/N$	All-Reduce + All-Gather 权重
ZeRO-2	优化器状态 + 梯度	$2\Psi + 14\Psi/N$	Reduce-Scatter + All-Gather
ZeRO-3	参数 + 梯度 + 优化器状态	$16\Psi/N$	前向 All-Gather，反向 Reduce-Scatter

$$\text{每卡状态} = \frac{16\Psi}{N}, \qquad N = 64: \ \frac{120}{64} \approx 1.9\ \text{GB}$$

以 7.5B 参数模型为例：ZeRO-3 阶段每个 GPU 的模型状态显存占用可从 120 GB 降至约 1.9 GB（假设使用 64 个 GPU）。

显存省下来了，但参数不再常驻：每次用到都得先收集，这正是 ZeRO-3 通信开销最大的原因。

ZeRO 三个阶段的显存对比

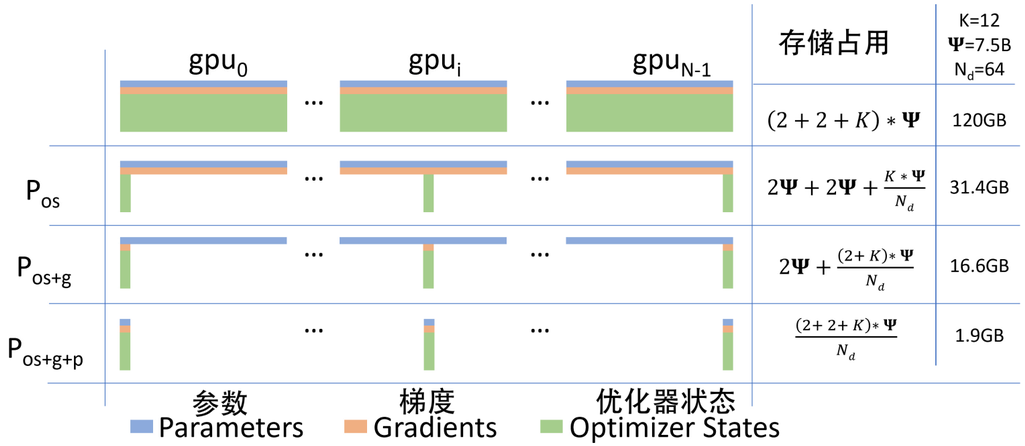
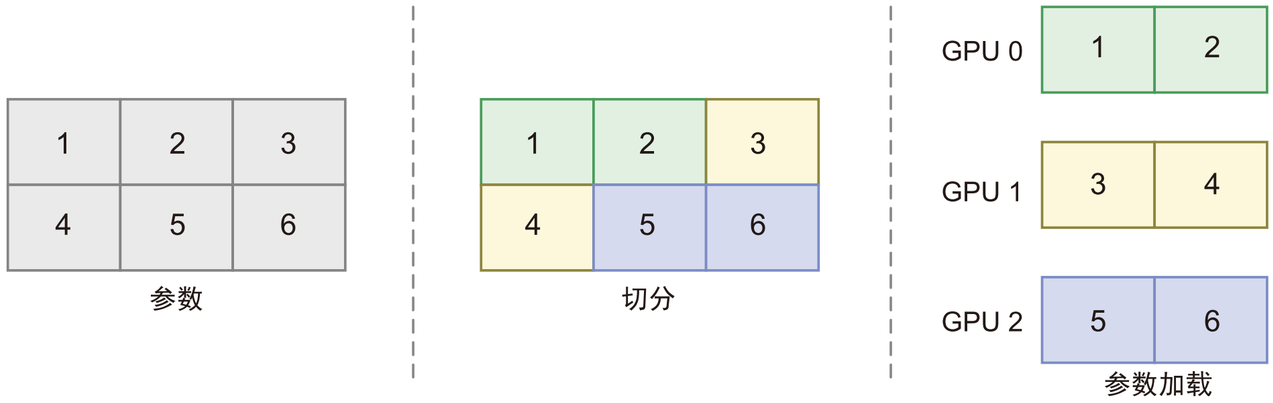


图 9.28　ZeRO 三阶段存储占用分析

图中按 7.5B 参数模型、64 张 GPU 给出各阶段的每卡占用：随切分粒度加深，参数、梯度与优化器状态依次被摊薄，直至全部状态都只保留 $1/N$。

横轴上的每一级都对应一次"多切一类状态、多一类通信"的交换，选择阶段本质上是选择愿意为显存付出多少带宽。

参数的切分方法



① 展平

将多维张量展平为一维向量

② 均分

按预设分区长度均匀切分

③ 记录

记录各分区的起止索引

所有 ZeRO 阶段都需根据参数分区创建对应的优化器状态分区，并维护反向传播产生的梯度分区。

图 9.29　ZeRO 参数切分策略

例如 2×3 大小的参数矩阵在 3 个 GPU 上的切分，按元素序列 $[a_{11}, a_{12}, a_{13}, a_{21}, a_{22}, a_{23}]$ 划分为 3 段，而非按矩阵行、列切分。

灵活性使其适用于任意模型架构，但 ZeRO-3 执行矩阵乘法时需实时收集参数分区（如 All-Gather），引入额外通信。

ZeRO-1

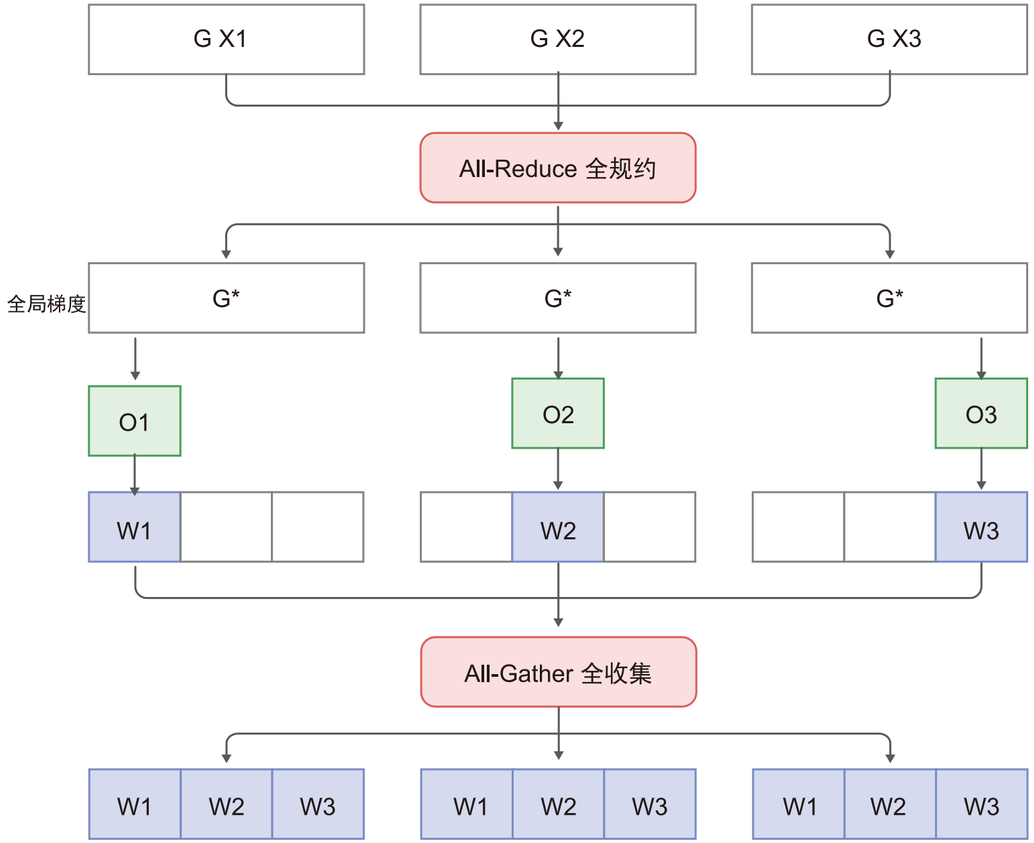


图 9.30　ZeRO-1 优化流程

- 每个 GPU 存储独立的优化器状态分区。
- 数据并行计算后，不同 GPU 上的模型副本产生的梯度不同，需通过 All-Reduce 操作进行均值全规约获得全局梯度。
- 随后，每个 GPU 根据全局梯度更新其本地维护的优化器状态分区所对应的模型权重。

$$\bar{\boldsymbol{g}} = \texttt{All-Reduce}\left(\boldsymbol{g}_0, \cdots, \boldsymbol{g}_{N-1}\right), \quad \boldsymbol{W}^{(i)} \leftarrow \boldsymbol{W}^{(i)} - \eta\, \bar{\boldsymbol{g}}^{(i)}$$

上标 (i) 表示第 i 个 GPU 负责的权重分区；全局梯度在每张卡上都相同，但只有对应分区的权重被更新。

更新完成后，通过 All-Gather 操作收集所有 GPU 上的权重分区，实现权重的一致性更新（见图 9.31）。

图 《大语言模型：训练、精调与对齐》图 9.30；第 9.3.1 节

全收集过程

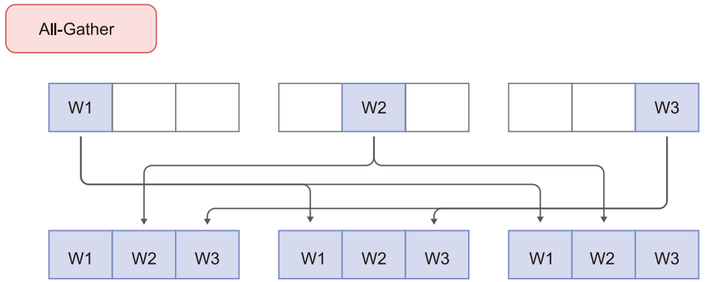


图 9.31　全收集通信算子

分区更新的好处是每个 GPU 只需为自己的那一片维护 FP32 优化器状态；代价是更新后每张卡只有部分权重是最新的。

$$\boldsymbol{W} \leftarrow \texttt{All-Gather}\left(\boldsymbol{W}^{(0)}, \boldsymbol{W}^{(1)}, \cdots, \boldsymbol{W}^{(N-1)}\right)$$

All-Gather 让所有设备重新获得完整的参数副本，为下一轮前向计算做好准备。

这次 All-Gather 是 ZeRO-1 相对普通数据并行新增的通信；到 ZeRO-3，参数本身就分区存放，反而不再需要这一步“额外”的同步。

图 《大语言模型：训练、精调与对齐》图 9.31；第 9.3.1 节

ZeRO-2

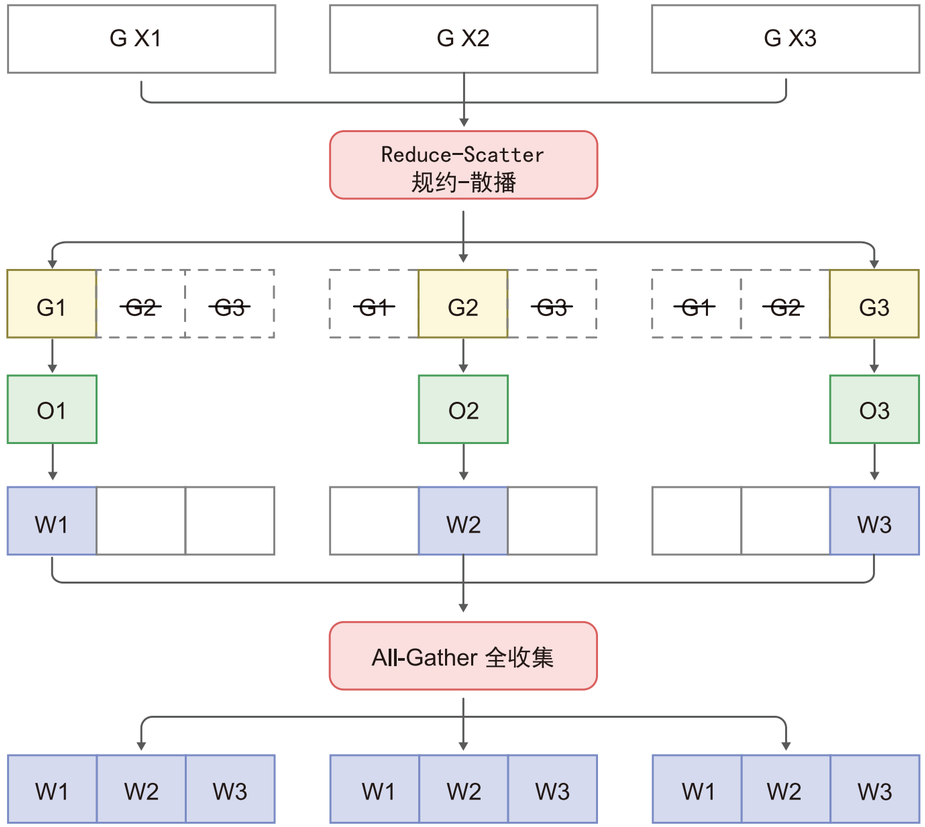


图 9.32　ZeRO-2 优化流程

ZeRO-1 的问题

梯度在各 GPU 上仍持有完整的副本，存在冗余。

ZeRO-2 的做法

将梯度全规约操作替换为规约-散播（Reduce-Scatter），使每个 GPU 仅获得并维护与其本地优化器状态分区相对应的部分梯度。

$$\hat{\boldsymbol{g}}^{(i)} = \texttt{Reduce-Scatter}\left(\boldsymbol{g}_0, \boldsymbol{g}_1, \cdots, \boldsymbol{g}_{N-1}\right)$$

设备 i 只拿回全局梯度的第 i 个分块 $\hat{\boldsymbol{g}}^{(i)}$，正好与它负责的优化器状态分区对应。

进一步分析，训练时通常会使用梯度累计技巧：ZeRO-1 在梯度累计步数里仅在最后一步做梯度全规约操作，ZeRO-2 在每个梯度累计步里都要做规约-散播操作。

ZeRO-3 的前向计算

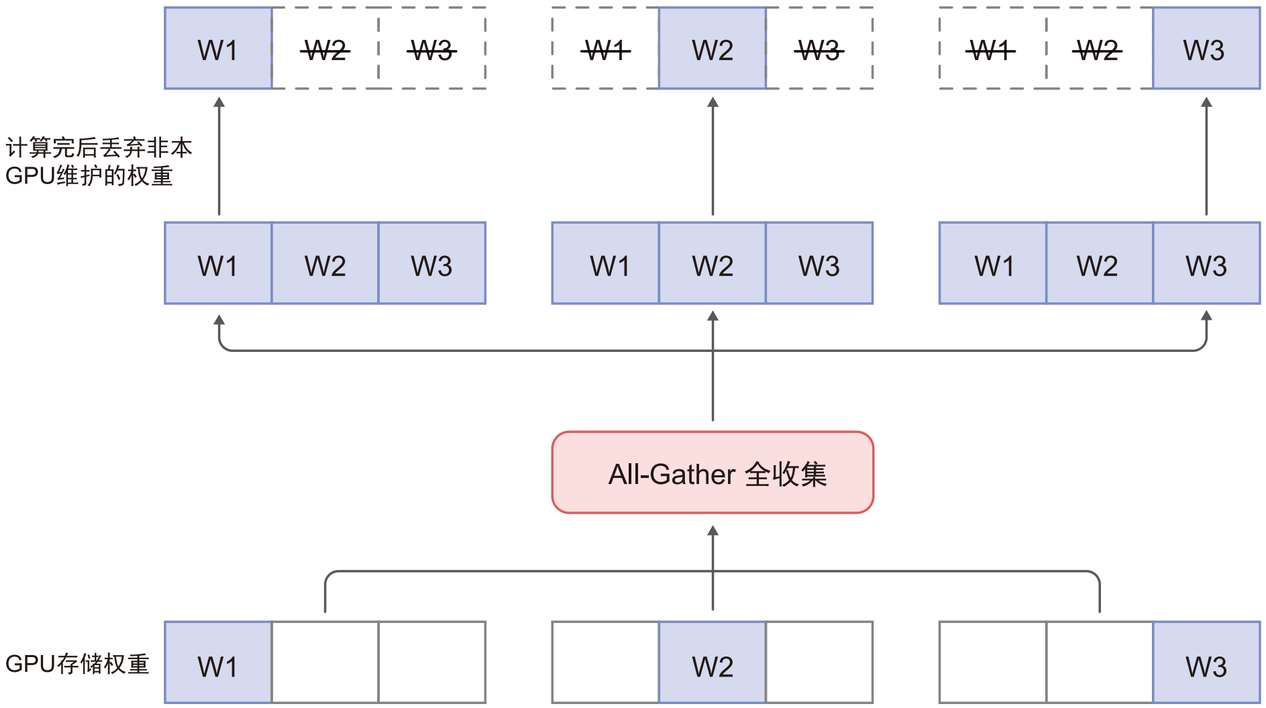


图 9.33　ZeRO-3 前向计算流程

- 前向计算时，每个 GPU 需通过 All-Gather 操作获取完整参数进行计算。
- 计算完成后释放非本地维护的参数分区，显存回到只存 $1/N$ 的状态。
- 参数按层“即取即用”，因此收集与释放的频率与层数成正比。

这一步是 ZeRO-3 最显著的额外开销：矩阵乘法无法在展平后的局部元素上完成，必须先拼接回完整张量。

反向传播时，梯度更新沿用 ZeRO-2 的方式：使用 Reduce-Scatter 操作，使每个 GPU 仅保留其负责维护的梯度分区（见图 9.34）。

ZeRO-3 的反向计算

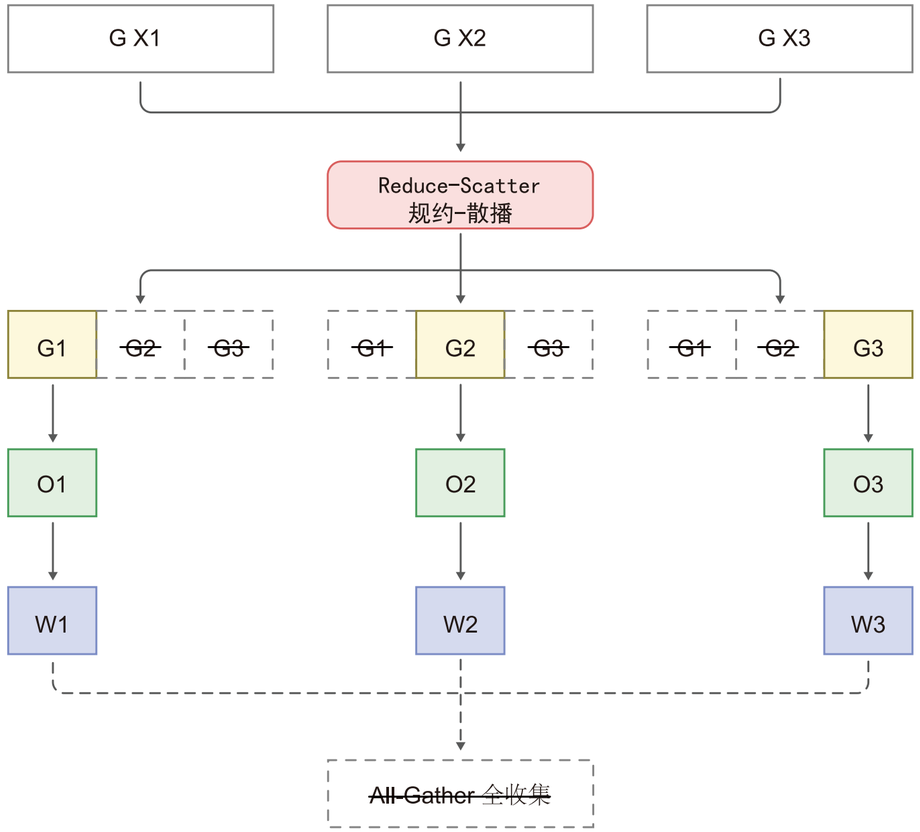


图 9.34　ZeRO-3 反向计算流程

① 前向

All-Gather 取参数，用完释放

② 反向

Reduce-Scatter 得本地梯度分区

③ 更新

就地更新本地参数分区，无需额外同步

参数已分区存储且更新在对应分区上进行，因此无需像 ZeRO-1 那样额外 All-Gather 同步权重。

> **小结**　ZeRO 以通信时间换显存空间；后续版本支持把部分状态卸载至 CPU 内存或外部存储。

图 《大语言模型：训练、精调与对齐》图 9.34；第 9.3.1 节

ZeRO 三个阶段的对比

基线

数据并行

每卡存完整参数、梯度、优化器状态。

All-Reduce 梯度

阶段 1

ZeRO-1

切优化器状态（动量与方差）。

All-Reduce + All-Gather 权重

阶段 2

ZeRO-2

再切梯度，与优化器分区对齐。

Reduce-Scatter + All-Gather

阶段 3

ZeRO-3

再切参数本身，全部状态各存 $1/N$。

前向 All-Gather，反向 Reduce-Scatter

$$\text{显存}\downarrow, \qquad \text{通信}\uparrow$$

每加深一级，就多切一类状态、多一类通信。ZeRO-3 省下的是参数本身的显存，付出的是每次前向都要重新收集参数。

这也解释了为什么章节小结会说：DeepSpeed 是对数据并行的一种扩展实现，由于其切分策略是通用的，在大型模型训练任务上计算效率不如以张量并行为主的 Megatron 框架。

9.4

小结

本节内容

系统设计的三条主线

四种并行算法的定位

成本与迭代速度

并行算法的核心理念，是把计算任务规模均等地划分成子任务，并使计算规模和通信负载同时保持均衡。

训练系统的三条主线

对于大型的分布式训练任务，其训练系统需要综合考虑硬件、模型结构和并行算法等特性，精心设计混合的并行策略，从而最大化利用计算资源并提高训练吞吐。

硬件

节点内带宽、节点间网络、显存容量

模型结构

层数、层内矩阵形状、注意力复杂度

并行算法

DP / TP / PP / CP 的组合与切分因子

吞吐

单位时间内处理更多词元

> **核心理念** 将计算任务规模均等地划分成子任务，并且使得计算规模和通信负载均衡。任何一维切得过细，都会让某一段链路成为新的瓶颈。

四种算法的适用范围

算法	优势	局限与适用条件
数据并行	实现简单、通用，可扩展到大量设备	状态冗余大；需 ZeRO 等技术削减
张量并行	切分层内参数，单卡显存显著下降	通信频繁，宜限制在节点内；推理场景也常用
流水线并行	设备间通信频率更低	存在气泡；通信性能较差的场景更有效
上下文并行	把长序列切到多卡	长文本训练的重要手段，也可加快推理的预填充

DeepSpeed

数据并行的扩展

由于切分策略是通用的，在大型模型训练任务上，计算效率不如以张量并行为主的 Megatron 框架。

上下文并行

可迁移的长序列手段

可以扩展到其他长序列注意力场景；在推理场景中可以加快预填充。

本章详解的四种并行算法构成 4D 并行的四个正交维度，实际系统按瓶颈选择切分哪一维、切多细。

训练效率的意义

大语言模型参数规模的急剧增加，为分布式训练领域带来快速发展的动力。在大型训练任务中，优化训练的成本是经济的；而更重要的是，减少耗时能为模型迭代研发带来竞争力。

成本

单位算力更值钱

集群规模越大、运行越久，MFU 每提高一个百分点所节省的机时与电费都相当可观。

时间

迭代周期缩短

一次预训练以周、月计；缩短耗时就意味着同一时间内能做更多轮实验与验证。

竞争力

研发节奏

训练系统的效率最终转化为模型迭代的速度，这是工程投入最直接的回报。

回看 从 9.1 的显存账本，到 9.2 的四种切分，再到 9.3 的状态分区，本章的每条线索都指向同一个目标：让同样的 GPU 在同样的时间里算出更多的梯度。

9.5

附录：专家并行*

MoE 把前馈层拆成多个专家：每台设备承载不同的专家，词元经由门控网络被派发到对应设备上计算。

本节内容

专家并行的部署方式

派发与合并：两次 All-to-All

负载不均衡与通信重叠

专家并行

在包含 MoE 模块的语言模型中，GShard 提出了专家并行：多设备部署结构相同的网络，但各设备的前馈层专家模块加载不同专家的参数。

- 每个设备处理独立的输入数据批次，因此在设备部署方式上与数据并行类似。
- 但核心差异在于：每个设备承载的是模型中不同的专家模型，而非完整的模型副本。
- 门控网络为每个词元选择 Top-K 个专家，决定了词元要去哪台设备。

对比

与数据并行的差别

数据并行中每张卡持有同一份完整参数；专家并行中每张卡只持有部分专家的参数，其余模块仍可能复制。

因此两种并行的“切分对象”不同：一个是数据，一个是前馈层中的专家集合。

关键 每个词元会通过门控网络被动态路由到 Top-K 个专家——路由结果在运行时才确定，这正是专家并行通信复杂的根源。

专家并行的数据流

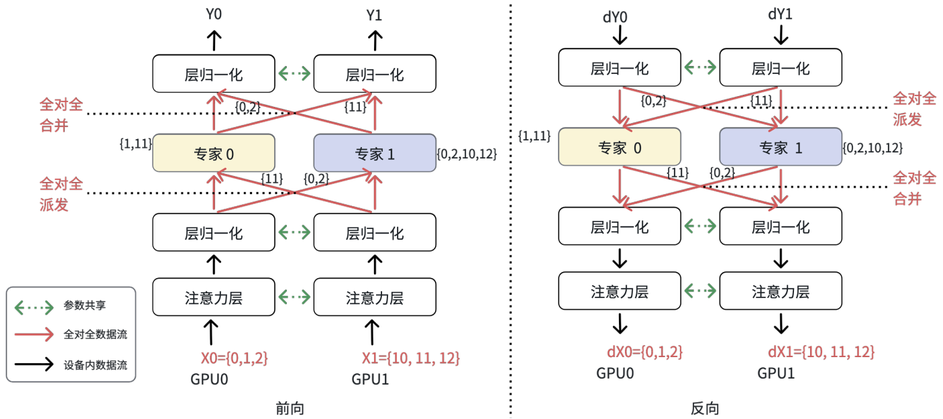


图 9.35　专家并行前后向数据流图

数据流可以概括为“派发—专家计算—合并”：词元先被送到持有目标专家的设备，专家算完后结果再被送回原设备，组合成最终的输出向量。

门控路由

每个词元选 Top-K 个专家

→

派发 Dispatch

All-to-All 送到专家所在设备

→

专家计算

各设备并行处理本地词元

→

合并 Combine

All-to-All 送回原设备求和

专家并行的两次全交换

这涉及两次关键的 All-to-All 通信操作：首先将词元派发到其目标专家所在的设备，然后在各专家处理完其分配到的词元后，将计算结果合并回原始设备。

阶段	通信原语	数据内容
派发 Dispatch	All-to-All	按目标专家分组后的词元隐藏状态
专家计算	无通信	各设备在本地词元上执行前馈层
合并 Combine	All-to-All	专家输出按来源设备回送并加权求和

$$\boldsymbol{y}_t = \sum_{k \in \text{Top-}K(t)} g_{t,k} \cdot \mathbf{Expert}_k\left(\boldsymbol{x}_t\right)$$

$g_{t,k}$ 为门控网络给出的权重。词元 t 只由被选中的 K 个专家计算，其余专家不参与，因此每台设备的工作量取决于被路由到它的词元数量。

门控路由导致的负载不均衡（不同专家分配到的词元数量不同）会直接引发通信量不均衡和各设备计算量不均衡的问题。

DualPipe 与通信重叠

为缓解专家并行中通信开销的影响，DeepSeek-V3 在其 MoE 模块中设计了双向流水线和计算-通信重叠机制，以更高效地隐藏通信延迟。

机制 1

双向流水线

从两个方向同时填充流水线，使同一时刻既有前向也有反向的计算在进行，从而压缩气泡。

机制 2

计算-通信重叠

把 All-to-All 的派发与合并安排在计算进行的同时，用计算时间掩盖通信时间。

> **同一思路** 9.1.3 节把“通信导致设备空闲”列为降低 MFU 的主因；这里给出的解法与流水线的 1F1B、1F1B1W 完全一致——不是消除通信，而是让它与计算并行。

双向流水线把前向与反向分到两个方向上同时填充，使“升温”与“收尾”两个气泡互相抵消；计算-通信重叠把 All-to-All 切成小块，与专家计算交替推进。两者叠加后，流水线在大部分时间保持满载。

结合 8 位精度训练等技术，DeepSeek-V3 在大型模型上进一步降低了计算量与通信量，说明显存、带宽与数值精度三者可以同时优化。

依据 《大语言模型：训练、精调与对齐》第 9.5 节

9.6

附录：训练-推理解耦架构*

PPO、GRPO 等算法需要在线采样：把推理集群与训练集群分开部署，才能让采样与训练各自跑在合适的并行策略上。

本节内容

解耦架构的动机

参数同步的开销

异步推理与离轨策略

训练与推理解耦的原因

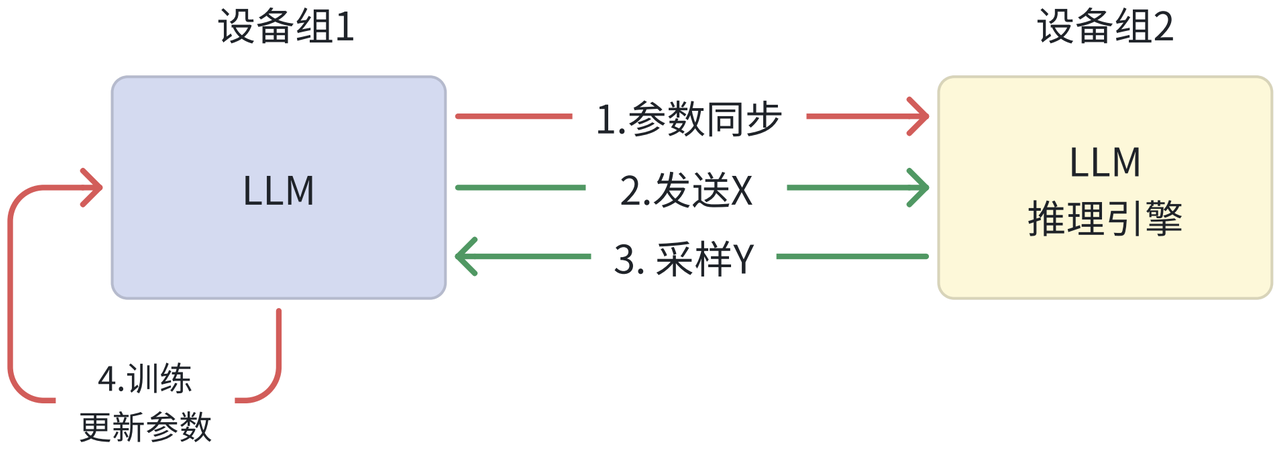


图 9.36　训练-推理解耦架构示意图

PPO 和 GRPO 等算法需要与环境在线交互。为此可采用训练-推理解耦架构：由专门的推理集群负责高效的数据采样。

推理集群

负责采样

与环境和提示词交互，生成 rollout；推理侧偏好低延迟、大并发的批处理。

训练集群

负责更新

在采样数据上计算策略梯度并更新参数；训练侧偏好高吞吐、状态分片。

解耦架构的参数同步

推理集群的参数需及时更新为训练产生的最新参数，以保证采样有效性，但这引入了参数同步开销。

训练更新
产生新参数 θ_{t+1}

→

参数同步
推送权重到推理集群

→

采样
用 θ_{t+1} 生成新轨迹

→

训练
用新轨迹算梯度

> **代价** 由于参数同步存在耗时，训练服务器的计算资源在等待推理结果时可能处于闲置状态。

这实质上是把数据并行里的“梯度同步”换成了“参数同步”：同步的数据量从梯度变成完整权重，频率也从每步一次变成每轮采样一次。此类架构的实现可参考 OpenRLHF 或 verl。

依据 《大语言模型：训练、精调与对齐》第 9.6 节

异步推理

为了提高系统整体的并行性与资源利用率，可以设计异步推理流程，例如 AReaL 所提出的方案。

同步推理

训练必须等待当前策略的采样完成，参数版本一致。

数据最新，但训练侧存在空等。

异步推理

训练与采样并行推进，训练使用稍早策略生成的数据。

利用率高，但数据滞后于当前策略。

> **算法要求** 在此异步模式下，训练过程使用的数据可能滞后于当前策略，因此算法需采用离轨策略优化方法进行训练。

进一步推广到更复杂的智能体系统，智能体与环境交互时可能涉及搜索网页、下载数据、运行代码等开放世界操作，单次交互的响应时间显著延长。这使得设计高效的异步推理机制，以及适配长延迟环境的离线策略训练算法，变得尤为关键。

思考题与实践题

下面 8 题均为动手实现，建议先用 2–4 个进程在本机跑通，再放大到多卡。实现时不必追求与生产框架同等的性能，重点是把通信原语的语义与数值一致性验证清楚。

1 实现 Reduce-Scatter：输入各进程的梯度张量，输出按 rank 分块规约的结果，并与 All-Reduce 后切片的结果逐元素比对。

2 实现非阻塞的 All-to-All：用 `isend/irecv` 拼出不带全局屏障的交换，比较它与阻塞版本在多轮叠加下的耗时。

3 实现分布式 Adam 优化器及梯度累积更新：参数与优化器状态分区存放，验证累积 K 步后更新与逐步更新等价。

4 实现 GQA 的张量并行前向与反向：KV 头在 TP 组内共享，验证前向输出与单卡结果一致、反向梯度能正确规约。

5 实现流水线并行的 1F1B：P 个阶段、M 个微批量，画出各阶段的时隙表，标出气泡并计算气泡占比。

6 实现基于 All-Reduce 的词表并行：输出层按词表切分，前向 All-Gather、反向 Reduce-Scatter，验证与不分词的 logits 一致；再实现嵌入层词表并行，嵌入表按词表切分，前向 All-Gather 拼出各词元向量、反向 Reduce-Scatter 分回梯度。

7 实现 ZeRO-3 训练：参数、梯度、优化器状态三分区，前向 All-Gather 取参数、用完释放，报告每卡显存与通信量。

8 实现专家并行，再进一步实现计算-通信重叠：用异步的 All-to-All 与专家计算重叠，比较重叠前后的吞吐率。

推荐使用 torch.distributed 模块进行实现。

依据 《大语言模型：训练、精调与对齐》第 9 章